

Generative AI — Assignment 1

Part 2: Job Postings Analysis — Role Categorization & Requirements Extraction (55 marks)

Dataset: `job_title_des.csv` — 2,277 job postings scraped from online job boards, with Job Title and Job Description.

Pipeline built with LangChain:

1. Load dataset into a DataFrame (`df.head(25)`)
2. Job category classification chain (few-shot, single label)
3. Requirements extraction chain — skills, education, experience as structured JSON
4. Apply both chains to every posting
5. Merge the new columns back into the original DataFrame

Backend is switchable between **Groq** (cloud) and **Ollama** (local — use this for the bonus run).

0. Setup

```
# Run once if needed
# !pip install -q langchain langchain-core langchain-groq langchain-ollama pandas tqdm

import os, json, re, time, warnings
import pandas as pd
from tqdm.auto import tqdm

warnings.filterwarnings("ignore")
pd.set_option("display.max_colwidth", 120)

# -----
# CHOOSE YOUR BACKEND: "groq" (cloud, fast) or "ollama" (local)
# -----
LLM_PROVIDER = "groq"

GROQ_MODEL = "llama-3.3-70b-versatile" # or "llama-3.1-8b-instant"
for speed
OLLAMA_MODEL = "llama3.2:3b" # pull first: ollama pull llama3.2:3b

if LLM_PROVIDER == "groq":
    from langchain_groq import ChatGroq
    if not os.environ.get("GROQ_API_KEY"):
        from getpass import getpass
        os.environ["GROQ_API_KEY"] = getpass("Enter GROQ_API_KEY: ")
    llm = ChatGroq(model=GROQ_MODEL, temperature=0, max_tokens=700)
```

```

else:
    from langchain_ollama import ChatOllama
    llm = ChatOllama(model=OLLAMA_MODEL, temperature=0,
num_predict=700)

print("Backend:", LLM_PROVIDER, "| model:", GROQ_MODEL if LLM_PROVIDER
== "groq" else OLLAMA_MODEL)

Backend: groq | model: llama-3.3-70b-versatile

```

Step 1: Load the Dataset

The file has an unnamed index column, so it's read with `index_col=0`. Columns are renamed to `Job_Title` / `Job_Description` to match the assignment's naming.

```

CSV_PATH = "job_title_des.csv"    # adjust path if needed

df_full = pd.read_csv(CSV_PATH, index_col=0)
df_full = df_full.rename(columns={"Job Title": "Job_Title",
                                "Job Description":
                                "Job_Description"})
df_full = df_full.reset_index(drop=True)
df_full.insert(0, "Job_ID", df_full.index)
df_full["Job_Description"] =
df_full["Job_Description"].astype(str).str.strip()

print("Full dataset shape:", df_full.shape)
print("Missing values:\n", df_full.isna().sum(), sep="")
print("\nUnique job titles:", df_full["Job_Title"].nunique())
display(df_full["Job_Title"].value_counts())
df_full.head(3)

```

Full dataset shape: (2277, 3)

Missing values:

Job_ID	0
Job_Title	0
Job_Description	0
dtype: int64	

Unique job titles: 15

Job_Title	
JavaScript Developer	166
Java Developer	161
Software Engineer	160
Node js developer	160
iOS Developer	159
PHP Developer	156

Flutter Developer	155
DevOps Engineer	155
Django Developer	152
Machine Learning	152
Backend Developer	147
Network Administrator	145
Database Administrator	139
Full Stack Developer	138
Wordpress Developer	132

Name: count, dtype: int64

	Job_ID	Job_Title \
0	0	Flutter Developer
1	1	Django Developer
2	2	Machine Learning

Job_Description

0 We are looking for hire experts flutter developer. So you are eligible this post then apply your resume.\nJob Types:...

1 PYTHON/DJANGO (Developer/Lead) - Job Code(PDJ - 04)\nStrong Python experience in API development (REST/RPC).\nExperi...

2 Data Scientist (Contractor)\n\nBangalore, IN\n\nResponsibilities\n\nWe are looking for a capable data scientist to j...

Assignment requirement: limit to the first 25 job postings

```
df = df_full.head(25).copy()
```

```
print("Working subset:", df.shape)
display(df[["Job_ID", "Job_Title"]])
print("\nSample description:\n")
print(df.iloc[0]["Job_Description"][:700])
```

Working subset: (25, 3)

	Job_ID	Job_Title
0	0	Flutter Developer
1	1	Django Developer
2	2	Machine Learning
3	3	iOS Developer
4	4	Full Stack Developer
5	5	Java Developer
6	6	Full Stack Developer
7	7	JavaScript Developer
8	8	DevOps Engineer
9	9	Software Engineer
10	10	Database Administrator
11	11	Machine Learning
12	12	Machine Learning
13	13	Software Engineer

14	14	Software Engineer
15	15	Java Developer
16	16	Wordpress Developer
17	17	iOS Developer
18	18	Database Administrator
19	19	DevOps Engineer
20	20	Database Administrator
21	21	Wordpress Developer
22	22	JavaScript Developer
23	23	Database Administrator
24	24	PHP Developer

Sample description:

We are looking for hire experts flutter developer. So you are eligible this post then apply your resume.

Job Types: Full-time, Part-time

Salary: ₹20,000.00 - ₹40,000.00 per month

Benefits:

Flexible schedule

Food allowance

Schedule:

Day shift

Supplemental Pay:

Joining bonus

Overtime pay

Experience:

total work: 1 year (Preferred)

Housing rent subsidy:

Yes

Industry:

Software Development

Work Remotely:

Temporarily due to COVID-19

Note on this dataset: all 15 distinct titles are software/IT roles (Flutter, Django, Java, DevOps, ML, DBA, ...), so the broad-domain classifier will legitimately return

Technology/IT for essentially every row. To make Step 2 actually informative, a second **sub-domain** classifier (Frontend / Backend / Mobile / Data & ML / DevOps & Cloud / Database / Full Stack / QA / Other) runs alongside it. The broad category is the graded deliverable; the sub-domain is the analytically useful one.

Step 2: Job Category Classification Task (10 marks)

A few-shot prompt returning one label from a fixed list, with **Other** as the fallback and a normaliser that snaps stray output back onto the allowed set.

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

CATEGORIES = ["Technology/IT", "Finance", "Marketing", "Healthcare",
              "Education", "Sales", "Operations", "Human Resources",
              "Other"]

category_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a job-posting classifier. Reply with EXACTLY ONE label  

     from this list and nothing "  

     "else: Technology/IT, Finance, Marketing, Healthcare, Education,  

     Sales, Operations, "  

     "Human Resources, Other. No explanation, no punctuation."),

    # --- few-shot examples ---
    ("human", "Job: Backend Developer.\nDescription: Build REST APIs  

     in Node.js, work with "  

     "MongoDB and Docker, own CI/CD pipelines.\nDomain  

     category:"),
    ("ai", "Technology/IT"),

    ("human", "Job: Financial Analyst.\nDescription: Prepare monthly  

     forecasts, variance analysis "  

     "and board reporting; strong Excel and IFRS knowledge.\n  

     nDomain category:"),
    ("ai", "Finance"),

    ("human", "Job: Digital Marketing Executive.\nDescription: Run  

     paid social campaigns, manage "  

     "SEO and email funnels, report on ROAS.\nDomain  

     category:"),
    ("ai", "Marketing"),

    ("human", "Job: Staff Nurse.\nDescription: Provide patient care in  

     the ICU, administer "  

     "medication, maintain clinical records.\nDomain  

     category:"),
    ("ai", "Healthcare"),

    # --- actual task ---
    ("human", "Given the following job title and description,  

     categorize the job into one of the "  

     "domains listed.\n\nJob: {title}\nDescription:  

     {description}\n\nDomain category:"),
    ])

category_chain = category_prompt | llm | StrOutputParser()

def normalise_category(raw: str) -> str:

```

```

text = (raw or "").strip().lower()
aliases = {
    "technology/it": "Technology/IT", "technology":
"Technology/IT", "it": "Technology/IT",
    "software": "Technology/IT", "engineering": "Technology/IT",
    "finance": "Finance", "marketing": "Marketing", "healthcare":
"Healthcare",
    "health care": "Healthcare", "education": "Education",
    "sales": "Sales",
    "operations": "Operations", "human resources": "Human
Resources", "hr": "Human Resources",
}
for k, v in sorted(aliases.items(), key=lambda x: -len(x[0])):
    if re.search(rf"\b{re.escape(k)}\b", text):
        return v
return "Other"

# ---- Optional second classifier: tech sub-domain (adds signal on
this all-IT dataset) ----
SUBDOMAINS = ["Frontend", "Backend", "Full Stack", "Mobile", "Data &
ML",
               "DevOps & Cloud", "Database", "QA & Testing", "Other"]

subdomain_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You classify technical job postings into one sub-domain. Reply
with EXACTLY ONE label from: "
     "Frontend, Backend, Full Stack, Mobile, Data & ML, DevOps &
Cloud, Database, QA & Testing, "
     "Other. Nothing else."),
    ("human", "Job: iOS Developer.\nDescription: Build native Swift
apps, publish to the App "
             "Store.\nSub-domain:"),
    ("ai", "Mobile"),
    ("human", "Job: Machine Learning.\nDescription: Train and deploy
predictive models in Python, "
             "TensorFlow, scikit-learn.\nSub-domain:"),
    ("ai", "Data & ML"),
    ("human", "Job: {title}.\nDescription: {description}\nSub-
domain:"),
])

subdomain_chain = subdomain_prompt | llm | StrOutputParser()

def normalise_subdomain(raw: str) -> str:
    text = (raw or "").strip().lower()
    for s in sorted(SUBDOMAINS, key=len, reverse=True):
        if s.lower() in text:
            return s

```

```

    if "machine learning" in text or "data scien" in text:
        return "Data & ML"
    return "Other"

import os, requests
from langchain_groq import ChatGroq

GROQ_MODEL = "openai/gpt-oss-20b"

r = requests.get("https://api.groq.com/openai/v1/models",
                  headers={"Authorization": f"Bearer {os.environ['GROQ_API_KEY']}"}, timeout=30)
available = sorted(m["id"] for m in r.json()["data"])
print("Available:", *available, sep="\n ")

if GROQ_MODEL not in available:
    GROQ_MODEL = next(m for m in available
                      if not any(k in m for k in ("whisper", "tts",
"guard", "embed")))
    print("\nFalling back to:", GROQ_MODEL)

llm = ChatGroq(model=GROQ_MODEL, temperature=0, max_tokens=700)

# rebind the chains to the new llm
category_chain      = category_prompt      | llm | StrOutputParser()
subdomain_chain     = subdomain_prompt     | llm | StrOutputParser()
requirements_chain  = requirements_prompt  | llm | StrOutputParser()

print("Active model:", llm.model_name)
print("Smoke test  :", llm.invoke("Reply with the single word:
OK").content.strip())

```

```

Available:
allam-2-7b
canopylabs/orpheus-arabic-saudi
canopylabs/orpheus-v1-english
groq/compound
groq/compound-mini
meta-llama/llama-prompt-guard-2-22m
meta-llama/llama-prompt-guard-2-86m
openai/gpt-oss-120b
openai/gpt-oss-20b
openai/gpt-oss-safeguard-20b
qwen/qwen3.6-27b
qwen/qwen3.8-27b
whisper-large-v3
whisper-large-v3-turbo

```

NameError

Traceback (most recent call

```

last)
Cell In[21], line 21
    19 category_chain      = category_prompt      | llm |
StrOutputParser()
    20 subdomain_chain    = subdomain_prompt    | llm |
StrOutputParser()
--> 21 requirements_chain = requirements_prompt | llm |
StrOutputParser()
    23 print("Active model:", llm.model_name)
    24 print("Smoke test  :", llm.invoke("Reply with the single word:
OK").content.strip())

```

NameError: name 'requirements_prompt' is not defined

```

# ---- Sample datapoint (Expected Output for Step 2) ----
sample = df.iloc[0]

```

```

def clean_output(text) -> str:
    """Strip reasoning traces / channel markers / code fences that
    gpt-oss and qwen models emit."""
    if not isinstance(text, str):
        text = getattr(text, "content", None) or str(text)

    # Complete <think>...</think> block, then a stray closing tag with
    no opener
    # (some models start streaming mid-reasoning), then an
    unterminated opener.
    text = re.sub(r"<think>.*?</think>", "", text, flags=re.DOTALL |
re.IGNORECASE)
    if re.search(r"</think>", text, re.IGNORECASE):
        text = re.split(r"</think>", text, flags=re.IGNORECASE)[-1]
        text = re.sub(r"<think>.*$", "", text, flags=re.DOTALL |
re.IGNORECASE)

    text = re.sub(r"<\|.*?\|>", "", text)           # <|channel|>, <|
message|>, ...
    text = re.sub(r"```(?:\w+)?|```", "", text)     # code fences
    return text.strip()

```

```

payload = {"title": sample["Job_Title"],
           "description": sample["Job_Description"][:4000]}

```

```

raw_cat = clean_output(category_chain.invoke(payload))
raw_sub = clean_output(subdomain_chain.invoke(payload))

```

```

print("Job Title      :", sample["Job_Title"])
print("Raw output     :", repr(raw_cat))

```

```
print("Predicted      :", normalise_category(raw_cat))
print("Sub-domain    :", normalise_subdomain(raw_sub))
```

```
Job Title      : Flutter Developer
Raw output     : 'Technology/IT'
Predicted      : Technology/IT
Sub-domain     : Mobile
```

Step 3: Requirements Extraction Task (30 marks)

A **single composite prompt** returning validated JSON with four fields — skills, education, experience, and an experience-in-years integer for easy filtering. Anything the posting doesn't mention comes back as "Not specified" (or an empty list), as required by the brief.

A regex + repair fallback handles models that wrap JSON in prose or code fences.

```
from pydantic import BaseModel, Field
from typing import List, Optional
from langchain_core.output_parsers import JsonOutputParser

class JobRequirements(BaseModel):
    skills: List[str] = Field(default_factory=list,
                              description="Specific skills, programming languages, frameworks or tools mentioned")
    education: str = Field(default="Not specified",
                           description="Minimum education required or preferred, e.g. \"Bachelor's degree in Computer Science\". Use 'Not specified' if absent.")
    experience: str = Field(default="Not specified",
                            description="Experience requirement as stated, e.g. '3+ years'. Use 'Not specified' if absent.")
    experience_years: Optional[float] = Field(default=None,
                                               description="Minimum years of experience as a number, or null if not stated")

req_parser = JsonOutputParser(pydantic_object=JobRequirements)

requirements_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You extract structured hiring requirements from job descriptions. You output ONLY valid "
     "JSON – no markdown fences, no commentary. Extract only what the posting actually states; "
     "never invent requirements. If a field is not mentioned, use \"Not specified\" for strings, "
     "an empty list for skills, and null for experience_years."),
```

```

        ("human",
         "Extract the required skills, education level and years of
         experience from the job "
         "description below.\n\n{format_instructions}\n\n"
         "Job Title: {title}\nJob Description:\n{description}\n\nJSON:"),
    ]).partial(format_instructions=req_parser.get_format_instructions())

requirements_chain = requirements_prompt | llm | StrOutputParser()

NOT_SPEC = "Not specified"

def _clean_str(val) -> str:
    if val is None:
        return NOT_SPEC
    if isinstance(val, list):
        val = ", ".join(str(v) for v in val if str(v).strip())
    text = str(val).strip()
    if not text or text.lower() in {"none", "null", "n/a", "na", "not
mentioned", "not stated", ""}:
        return NOT_SPEC
    return text

def parse_requirements(raw: str) -> dict:
    """Robustly turn the model's reply into the four requirement
    fields."""
    default = {"skills": [], "education": NOT_SPEC,
               "experience": NOT_SPEC, "experience_years": None}
    if not raw:
        return default

    text = re.sub(r"```(?:json)?|```", "", raw).strip()
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        return default
    try:
        data = json.loads(match.group(0))
    except json.JSONDecodeError:
        try:
            # last-ditch repair:
            trailing commas
            data = json.loads(re.sub(r",\s*([}\]])", r"\1",
match.group(0)))
        except json.JSONDecodeError:
            return default

    skills = data.get("skills", [])
    if isinstance(skills, str):
        skills = [s.strip() for s in re.split(r"[,;\n]", skills) if
s.strip()]
    seen, clean_skills = set(), []

```

```

    for s in skills:
        s = str(s).strip().strip(".")
        if s and s.lower() not in seen and s.lower() not in {"not
specified", "none"}:
            seen.add(s.lower())
            clean_skills.append(s)

    experience = _clean_str(data.get("experience"))
    years = data.get("experience_years")
    try:
        years = float(years) if years is not None else None
    except (TypeError, ValueError):
        years = None
    if years is None and experience != NOT_SPEC:                # backfill
from the text
        m = re.search(r"(\d+(?:\.\d+)?)\s*\+?\s*(?:to|-|-)?\s*\d*\s*years?", experience, re.I)
        if m:
            years = float(m.group(1))

    return {"skills": clean_skills,
            "education": _clean_str(data.get("education")),
            "experience": experience,
            "experience_years": years}

# ---- Sample datapoint (Expected Output for Step 3) ----
raw_req = requirements_chain.invoke({"title": sample["Job_Title"],
                                     "description":
sample["Job_Description"][:6000]})
parsed = parse_requirements(raw_req)

print("Raw output:\n", raw_req.strip()[:700], "\n")
print(json.dumps(parsed, indent=2, ensure_ascii=False))

Raw output:
{"skills":["Flutter"],"education":"Not specified","experience":"total
work: 1 year (Preferred)","experience_years":1}

{
  "skills": [
    "Flutter"
  ],
  "education": "Not specified",
  "experience": "total work: 1 year (Preferred)",
  "experience_years": 1.0
}

```

Step 4: Apply the Chain to Each Job Posting (10 marks)

`safe_invoke` retries with exponential backoff so one rate-limit response doesn't abort the run. Three LLM calls per posting: category, sub-domain, requirements.

```
TEXT_LIMIT = 6000      # characters of the description fed to the LLM
MAX_RETRIES = 3
SLEEP       = 0.0       # raise to ~1.5 if Groq rate-limits you

def safe_invoke(chain, payload, default=""):
    for attempt in range(MAX_RETRIES):
        try:
            return chain.invoke(payload)
        except Exception as e:
            wait = 2 ** attempt
            print(f" ! {type(e).__name__}: {str(e)[:90]} - retry in {wait}s")
            time.sleep(wait)
    return default

def process_posting(row: pd.Series) -> dict:
    payload = {"title": row["Job_Title"], "description":
row["Job_Description"][:TEXT_LIMIT]}

    cat_raw = safe_invoke(category_chain, payload)
    sub_raw = safe_invoke(subdomain_chain, payload)
    req_raw = safe_invoke(requirements_chain, payload)
    req      = parse_requirements(req_raw)

    return {
        "Job_ID":          row["Job_ID"],
        "Predicted_Category": normalise_category(cat_raw),
        "Tech_Subdomain":   normalise_subdomain(sub_raw),
        "Required_Skills":  req["skills"],
        "Education_Required": req["education"],
        "Experience_Required": req["experience"],
        "Experience_Years":  req["experience_years"],
    }

def run_pipeline(frame: pd.DataFrame) -> pd.DataFrame:
    records = []
    for _, row in tqdm(frame.iterrows(), total=len(frame),
desc="Processing postings"):
        records.append(process_posting(row))
        if SLEEP:
            time.sleep(SLEEP)
    return pd.DataFrame(records)
```

```

results_df = run_pipeline(df) # <- the 25-posting run
print("Results shape:", results_df.shape)
results_df.head()

{"model_id": "143971cf3b01466faf45cfb9444839b2", "version_major": 2, "version_minor": 0}

```

Results shape: (25, 7)

	Job_ID	Predicted_Category	Tech_Subdomain \
0	0	Technology/IT	Mobile
1	1	Technology/IT	Backend
2	2	Technology/IT	Data & ML
3	3	Technology/IT	Mobile
4	4	Technology/IT	Full Stack

Required_Skills \

0	[Flutter]
1	[Python, Django, Flask, REST, RPC, Linux, SQL, JSON, PyUnit]
2	[Python, Java, Spark, PyTorch, TensorFlow, Keras, big data]
3	[Objective-C, Cocoa Touch, Core Data, Core Animation, Core Graphics, Core Text, networking, concurrency, threading, ...]
4	[React, React Native, Redux, Angular, Vue, JavaScript, HTML, CSS, RESTful APIs, HTTP networking, MVC, Object-Oriented...]

Education_Required \

0	Not specified
1	Not specified
2	Graduate or M.Sc. in Computer Science, Mathematics or equivalent, preferably in Machine Learning
3	Not specified
4	Not specified

Experience_Required \

0	total work: 1 year (Preferred)
1	Not specified
2	At least 3 years of hands-on development of complex Machine Learning models using modern frameworks and tools, ideal...

```
3                                     iOS
experience, published one or more iOS apps in the app store
4                                     5+ year hands-on experience web
development, 2+ year recent experience working react
```

```
Experience_Years
0          1.0
1         NaN
2          3.0
3         NaN
4          5.0
```

Step 5: Update the DataFrame with New Columns (5 marks)

```
final_df = df.merge(results_df, on="Job_ID", how="left")

cols = ["Job_ID", "Job_Title", "Job_Description",
        "Predicted_Category", "Tech_Subdomain", "Required_Skills",
        "Education_Required", "Experience_Required",
        "Experience_Years"]
final_df = final_df[cols]

print("Final dataframe shape:", final_df.shape)
final_df[["Job_ID", "Job_Title", "Predicted_Category",
          "Tech_Subdomain",
          "Required_Skills", "Education_Required",
          "Experience_Required"]]
```

Final dataframe shape: (25, 9)

	Job_ID	Job_Title	Predicted_Category	Tech_Subdomain
0	0	Flutter Developer	Technology/IT	Mobile
1	1	Django Developer	Technology/IT	Backend
2	2	Machine Learning	Technology/IT	Data & ML
3	3	iOS Developer	Technology/IT	Mobile
4	4	Full Stack Developer	Technology/IT	Full Stack
5	5	Java Developer	Technology/IT	Full Stack
6	6	Full Stack Developer	Technology/IT	Full Stack
7	7	JavaScript Developer	Technology/IT	Full Stack

8	8	DevOps Engineer	Technology/IT	DevOps & Cloud
9	9	Software Engineer	Technology/IT	Backend
10	10	Database Administrator	Technology/IT	Database
11	11	Machine Learning	Technology/IT	Data & ML
12	12	Machine Learning	Technology/IT	Data & ML
13	13	Software Engineer	Technology/IT	Full Stack
14	14	Software Engineer	Technology/IT	Full Stack
15	15	Java Developer	Technology/IT	Frontend
16	16	Wordpress Developer	Technology/IT	Full Stack
17	17	iOS Developer	Technology/IT	Mobile
18	18	Database Administrator	Technology/IT	Database
19	19	DevOps Engineer	Technology/IT	DevOps & Cloud
20	20	Database Administrator	Technology/IT	Database
21	21	Wordpress Developer	Technology/IT	Full Stack
22	22	JavaScript Developer	Technology/IT	Full Stack
23	23	Database Administrator	Technology/IT	Database
24	24	PHP Developer	Technology/IT	Full Stack

Required_Skills \

0

[Flutter]

1

[Python, Django, Flask, REST, RPC, Linux, SQL, JSON, PyUnit]

2

[Python, Java, Spark, PyTorch, TensorFlow, Keras, big data]

3 [Objective-C, Cocoa Touch, Core Data, Core Animation, Core Graphics, Core Text, networking, concurrency, threading, ...]

4 [React, React Native, Redux, Angular, Vue, JavaScript, HTML, CSS, RESTful APIs, HTTP networking, MVC, Object-Oriented...]

5 [C#, NET, NET Core, HTML5, CSS3, MsSQL, MySQL, ReactJS, WSDL, Soap, Restful, web services, relational databases, dat...]

6

[]
 7 [ReactJS, NodeJS, Azure Functions, GraphQL, HTML5, CSS3, JavaScript, REST]
 8 [Bash, Ruby, Python, Java, Scripting, Puppet, Chef, Cloudify, CFEngine, Cobbler, Foreman, PHP, Linux, Windows OS, Ur...
 9 [REST API, C/C++ for Linux/Unix, Python, Go, Git, Gerrit, Jenkins, configuration and deployment of large systems on ...
 10
 11 [TensorFlow, OpenCV, Python, Artificial Neural Network (ANN), Deep Learning, Image Processing, PyTorch, Amazon Web S...
 12 [Java/Scala, C++, Perl/Python, R, Matlab, predictive modelling, predictive software development, distributed systems...
 13
 14 [Agile, unit tests, integration tests, source code management, Azure, Google Cloud, AWS, Object-oriented design, Jav...
 15 [modern Javascript (ES6+), Node, HTML5, CSS3, SCSS, Vue or similar web frameworks (React, Angular), REST web service...
 16
 17 [iOS Development, Native iOS development, PhoneGap, Ionic, Titanium, Database knowledge, Requirement Gathering, Clie...
 18 [HPE Oracle database environment, Oracle database administration, Database middleware technology, Logical physical d...
 19
 20 [Oracle Database, SQL, Performance Tuning, Replication, Distributed Data, Linux, Database Recovery, WebLogic, Databa...
 21 [CSS3, JavaScript, HTML5, jQuery, Git, Mercurial, SVN, Chrome Inspector, Firebug, WordPress themes, WordPress plugin...
 22 [JavaScript, Python/Flask, PostgreSQL, Linux, Docker, Git, Cloud Services, APIs, UI]
 23
 24
 []

Education_Required \

0

Not specified

1

Not specified

2 Graduate or M.Sc. in Computer Science, Mathematics or equivalent, preferably in Machine Learning

3

Not specified

4

Not specified
5 Bachelor's Degree in Computer Science, Information Systems, or
related field, or combination of education and equiva...
6
Not specified
7
Any graduation, and Any PG and Any Doctorate
8
Not specified
9 Minimum of BS or MS; computer
engineering, computer science or related technical field.
10
Not specified
11
Not specified
12 Masters and/or PhD in Computer Science, Machine Learning,
Operational research, Statistics or in a highly quantitati...
13
Not specified
14 Bachelor's degree or higher education level, or its foreign
equivalent, in Computer Science, Computer Information Sc...
15
B.S. Degree in Computer Science or equivalent experience
16
Not specified
17 BE(Computers/
Information Technology), MCA, BCA, BSc (Computer Science)
18 Bachelor's degree in
Computer Science or Information Technology related field
19
Not specified
20
Bachelor's degree
21
Bachelor's degree in Computer Science or a similar field.
22
Not specified
23
Not specified
24
Not specified

Experience_Required

0
total work: 1 year (Preferred)
1
Not specified
2 At least 3 years of hands-on development of complex Machine

```

Learning models using modern frameworks and tools, ideal...
3
experience, published one or more iOS apps in the app store
4
5+ year hands-on experience web
development, 2+ year recent experience working react
5
minimum of two years of
experience using the tools and technologies noted above.
6
Not specified
7
3 - 8 years
8
Not specified
9
Minimum 7 years of software development experience
10
Not specified
11
2 - 4 years
12
Experience in predictive
modelling and analysis, predictive software development
13
Not specified
14 Total Work Experience: 6+ years (software development), 4 years
minimum, Product Experience: 2 years minimum, 4+ yea...
15
5+ years of software development experience
16
Not specified
17
1 to 2 years in iOS Development
18
10+ years
19
Not specified
20
2+ years
21
4 to 8 Years
22
Not specified
23
Not specified
24
Not specified

# ---- Full record view, matching the example JSON in the assignment
----
rec = final_df.iloc[0].to_dict()
rec["Job_Description"] = rec["Job_Description"][:250] + " ..."

```

```
[excerpt]"
print(json.dumps(rec, indent=2, ensure_ascii=False))

{
  "Job_ID": 0,
  "Job_Title": "Flutter Developer",
  "Job_Description": "We are looking for hire experts flutter
developer. So you are eligible this post then apply your resume.\nJob
Types: Full-time, Part-time\nSalary: ₹20,000.00 - ₹40,000.00 per
month\nBenefits:\nFlexible schedule\nFood allowance\nSchedule:\nDay
shift\nSuppleme ... [excerpt]",
  "Predicted_Category": "Technology/IT",
  "Tech_Subdomain": "Mobile",
  "Required_Skills": [
    "Flutter"
  ],
  "Education_Required": "Not specified",
  "Experience_Required": "total work: 1 year (Preferred)",
  "Experience_Years": 1.0
}
```

Spot-checks and sanity summary

```
print("Category distribution:")
display(final_df["Predicted_Category"].value_counts())

print("Tech sub-domain distribution:")
display(final_df["Tech_Subdomain"].value_counts())

print("Fields returned as 'Not specified':")
display(pd.Series({
  "Education": (final_df["Education_Required"] == "Not
specified").sum(),
  "Experience": (final_df["Experience_Required"] == "Not
specified").sum(),
  "Skills (empty)":
final_df["Required_Skills"].apply(len).eq(0).sum(),
}))

print("\nSkills per posting – mean %.1f, min %d, max %d"
      % (final_df["Required_Skills"].apply(len).mean(),
        final_df["Required_Skills"].apply(len).min(),
        final_df["Required_Skills"].apply(len).max()))

print("\nTop 20 skills across the 25 postings:")
all_skills =
final_df["Required_Skills"].explode().dropna().str.strip().str.title()
display(all_skills.value_counts().head(20))
```

Category distribution:

```
Predicted_Category
Technology/IT      25
Name: count, dtype: int64
```

Tech sub-domain distribution:

```
Tech_Subdomain
Full Stack      10
Database        4
Mobile          3
Data & ML       3
Backend         2
DevOps & Cloud  2
Frontend        1
Name: count, dtype: int64
```

Fields returned as 'Not specified':

```
Education      14
Experience     10
Skills (empty)  7
dtype: int64
```

Skills per posting – mean 9.6, min 0, max 25

Top 20 skills across the 25 postings:

```
Required_Skills
Python      5
Html5       5
Java        4
Linux       4
Javascript  4
Css3        4
Git         3
Pytorch     2
Apis        2
Tensorflow  2
Reactjs     2
Css         2
Keras       2
Sql         2
Jenkins     2
Rest        2
Big Data    1
Json        1
Pyunit      1
Spark       1
Name: count, dtype: int64
```

```
# ---- Manual spot-check: does the extraction match the source text?
----
```

```
for i in [0, 5, 12]:
    r = final_df.iloc[i]
    print("=" * 90)
    print("TITLE      :", r["Job_Title"])
    print("CATEGORY   :", r["Predicted_Category"], "|",
r["Tech_Subdomain"])
    print("SKILLS      :", ", ".join(r["Required_Skills"]) or "-")
    print("EDUCATION   :", r["Education_Required"])
    print("EXPERIENCE  :", r["Experience_Required"])
    print("-" * 90)
    print("SOURCE EXCERPT:\n", r["Job_Description"][:500], "...\\n")
```

```
=====
=====
```

```
TITLE      : Flutter Developer
CATEGORY   : Technology/IT | Mobile
SKILLS     : Flutter
EDUCATION  : Not specified
EXPERIENCE : total work: 1 year (Preferred)
```

```
-----
-----
```

SOURCE EXCERPT:

We are looking for hire experts flutter developer. So you are eligible this post then apply your resume.

Job Types: Full-time, Part-time

Salary: ₹20,000.00 - ₹40,000.00 per month

Benefits:

Flexible schedule

Food allowance

Schedule:

Day shift

Supplemental Pay:

Joining bonus

Overtime pay

Experience:

total work: 1 year (Preferred)

Housing rent subsidy:

Yes

Industry:

Software Development

Work Remotely:

Temporarily due to COVID-19 ...

```
=====
=====
```

```
TITLE      : Java Developer
CATEGORY   : Technology/IT | Full Stack
SKILLS     : C#, NET, NET Core, HTML5, CSS3, MsSQL, MySQL, ReactJS,
```

WSDL, Soap, Restful, web services, relational databases, data access, database queries, software design patterns, MVC architecture, Microsoft IIS, SYSPRO ERP

EDUCATION : Bachelor's Degree in Computer Science, Information Systems, or related field, or combination of education and equivalent work experience.

EXPERIENCE : minimum of two years of experience using the tools and technologies noted above.

SOURCE EXCERPT:

Software Developer - Integration*

Immediate Opening!*

A dynamic Akron / Cleveland area company is looking for an experienced software developer. The Integration Developer designs, develops, tests, maintains and enhances multiple systems serving internal users as well as our suppliers and customers. The Integration Developer interacts with IT and other internal business partners to deliver systems that advance company strategy and improve the enterprise capability of IT.

Candidates need to demons ...

=====

TITLE : Machine Learning

CATEGORY : Technology/IT | Data & ML

SKILLS : Java/Scala, C++, Perl/Python, R, Matlab, predictive modelling, predictive software development, distributed systems, grid computing, machine learning, data mining, statistical techniques

EDUCATION : Masters and/or PhD in Computer Science, Machine Learning, Operational research, Statistics or in a highly quantitative field

EXPERIENCE : Experience in predictive modelling and analysis, predictive software development

SOURCE EXCERPT:

Excited by Big Data, Machine Learning and Predictive Software?

Interested in creating new state-of-the-art solutions using Machine Learning and Data Mining techniques on Terabytes of Data?

At Amazon, we are developing state-of-the-art large-scale Machine Learning Services and Applications on the Cloud involving Terabytes of data. We work on applying predictive technology to a wide spectrum of problems in areas such as Amazon Retail, Seller Services, Customer Service, Alexa, Chatbots and so on. W ...

---- Save outputs ----

final_df.to_csv("part2_job_results_25.csv", index=False)

with open("part2_job_results_25.json", "w", encoding="utf-8") as f:

```
    json.dump(final_df.to_dict(orient="records"), f, indent=2,
ensure_ascii=False)
```

```
print("Saved: part2_job_results_25.csv / .json")
```

```
Saved: part2_job_results_25.csv / .json
```

Bonus (optional, +10 marks): Run on ALL 2,277 postings

Switch `LLM_PROVIDER` to `"ollama"` and restart before enabling this — $2,277 \times 3$ calls will exhaust Groq's free tier. Progress is checkpointed to JSONL every `SAVE_EVERY` postings so an interrupted run resumes instead of restarting.

```
RUN_FULL_DATASET = False          # flip to True when you're ready
CHECKPOINT       = "part2_full_checkpoint.jsonl"
SAVE_EVERY       = 25

if RUN_FULL_DATASET:
    done = set()
    if os.path.exists(CHECKPOINT):
        with open(CHECKPOINT, encoding="utf-8") as f:
            done = {json.loads(line)["Job_ID"] for line in f if
line.strip()}
        print(f"Resuming — {len(done)} postings already processed")

    todo = df_full[~df_full["Job_ID"].isin(done)]
    buffer = []

    with open(CHECKPOINT, "a", encoding="utf-8") as f:
        for i, (_, row) in enumerate(tqdm(todo.iterrows(),
total=len(todo), desc="Full dataset"), 1):
            buffer.append(process_posting(row))
            if i % SAVE_EVERY == 0 or i == len(todo):
                for r in buffer:
                    f.write(json.dumps(r, ensure_ascii=False) + "\n")
                f.flush()
                buffer = []

    all_results = pd.read_json(CHECKPOINT, lines=True)
    full_final = df_full.merge(all_results, on="Job_ID", how="left")
    full_final.to_csv("part2_job_results_full.csv", index=False)

    print("\nFull dataset processed:", full_final.shape)
    display(full_final["Predicted_Category"].value_counts())
    display(full_final["Tech_Subdomain"].value_counts())
    display(pd.crosstab(full_final["Job_Title"],
full_final["Tech_Subdomain"])))
```

```
else:  
    print("Set RUN_FULL_DATASET = True to attempt the bonus.")
```

Set RUN_FULL_DATASET = True to attempt the bonus.

Summary of what was built

Step	Deliverable	Where
1	Dataset loaded into a DataFrame, limited to <code>df.head(25)</code>	Step 1
2	Few-shot domain classifier + label normaliser (+ tech sub-domain classifier)	Step 2
3	Single composite extraction chain → skills / education / experience as validated JSON	Step 3
4	<code>process_posting</code> loop with retry and <code>Not specified</code> handling	Step 4
5	<code>Predicted_Category</code> , <code>Required_Skills</code> , <code>Education_Required</code> , <code>Experience_Required</code> merged into the original DataFrame	Step 5
Bonus	Checkpointed runner for all 2,277 postings	Bonus cell